DECA Lab Logbook — Spring Term

Lab 1 | Task 6: CMP and AND Instructions

Task 6: CMP and AND Instructions

Objectives

- How CMP differs from SUB at hardware level, and why
- How the AND instruction is implemented in hardware and how it relates to Boolean AND

1. CMP vs SUB

The instruction CMP performs a subtraction internally, but its purpose is to update flags rather than to write a data result.

1: Identify which piece of hardware is responsible for the difference in behaviour between SUB and CMP. In particular, discuss the significance of the signal DPDECODE.WEN1.

The difference is controlled by DPDECODE.WEN1, the register-file write-enable signal. SUB and CMP execute an identical ADDSUB computation; the only distinction is:

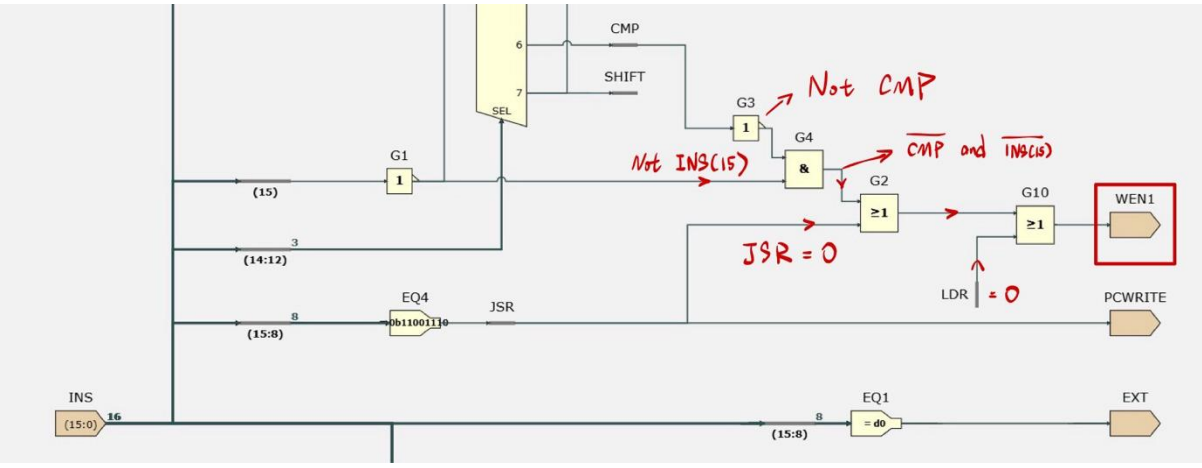
- SUB: WEN1 = 1 — the result is written to the destination register.
- CMP: WEN1 = 0 — no register is written. The subtraction still executes to update the condition flags (FlagN, FlagZ, FlagC, FlagV), but the result is discarded.

2: Inspect the logic in the dpdecode sheet that drives DPDECODE.WEN1. You may assume that, for this question, JSR = 0 and LDR = 0.

With JSR = 0 and LDR = 0, the WEN1 logic simplifies to:

```
WEN1  =  NOT INS(15)  AND  NOT CMP
```

INS(15) = 0 identifies the ALU instruction class. NOT CMP is 0 only for the CMP opcode, making WEN1 = 1 for all ALU instructions except CMP.



3: Write a truth table that shows the value of WEN1 for each of the eight ALU instructions shown in Figure 7. Explain what this table shows about when the ALU result is written back to the register file and when it is not.

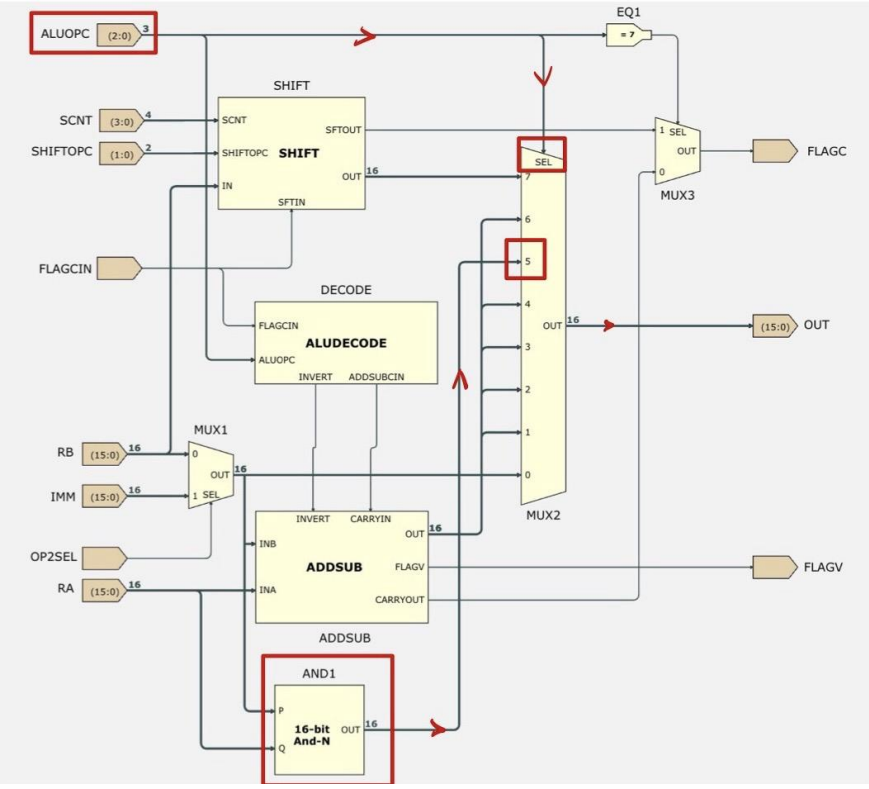
Instruction	WEN1	Effect
MOV	1	Result written to Ra
ADD	1	Result written to Rc
SUB	1	Result written to Rc
ADC	1	Result written to Rc
SBC	1	Result written to Rc
AND	1	Result written to Rc
CMP	0	No register written — flags updated only
SHIFT	1	Result written to Ra

CMP is the only instruction with WEN1 = 0. All other ALU instructions commit their result to the register file. CMP is designed exclusively to set condition flags by performing a subtraction; retaining the numerical result is neither needed nor intended.

2. AND instruction

1: Identify the hardware that implements the AND instruction: which input signals feed the logic block, and through which mux selection does the ALU choose the AND result?

The AND instruction is implemented by block AND1, a 16-bit bitwise AND gate in the ALU sheet. Its inputs are RA (register Ra from DOUT2) and the output of MUX1 (either RB or the sign-extended immediate, depending on OP2SEL). When ALUOPC = 5, the output MUX selects AND1's result as the ALU output; for all other opcodes the AND1 output is not selected.



2: Explain how this hardware behaviour corresponds to the Boolean AND operation on the two operands.

The 16-bit AND gate applies a 1-bit Boolean AND independently to each of the 16 bit positions: OUT[i] = RA[i] AND OP2[i]. This is exactly a bitwise AND — the Boolean AND truth table is applied in parallel across all 16 bit pairs simultaneously.

Extension questions

1: If you wanted to implement additional logic functions (for example OR, XOR, etc.), which blocks would you need to add or modify? How would you change the existing hardware and the width of ALUOPC to support these new operations?

To add OR and XOR: add a 16-bit OR gate and a 16-bit XOR gate as parallel functional blocks, with RA and MUX1 as inputs. Extend ALUOPC from 3 bits to 4 bits to address more than 8 operations. Expand the output MUX to include the two new inputs. Update ALU.DECODE for the new opcodes; INVERT and ADDSUBCIN can be don't-cares since ADDSUB.OUT is unused for these operations.

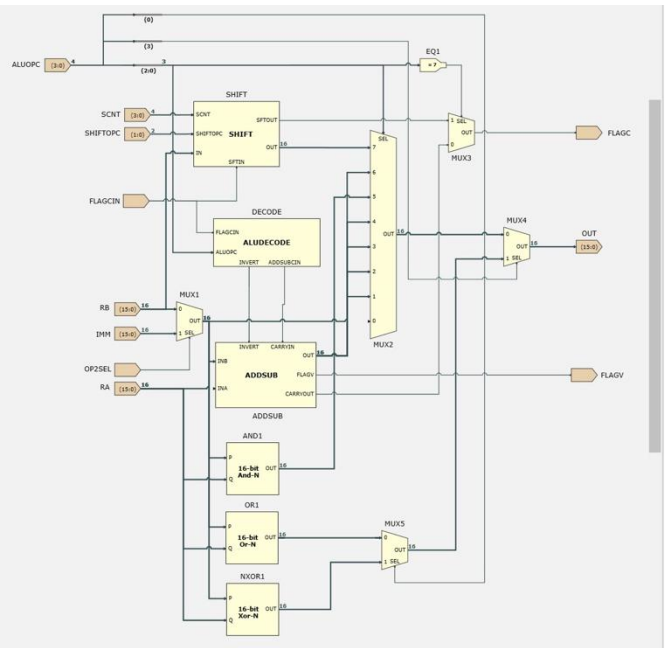
2: Design a new ALU sheet that extends the current design to perform additional logic functions as well as the existing ones. You will need to change the width of ALUOPC. This design does not need to integrate with the EEP1 datapath.

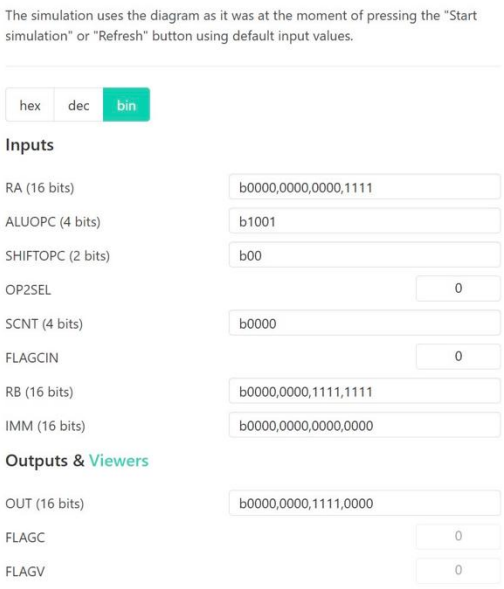
ALUOPC is widened to 4 bits to support up to 16 operations. The original 8-to-1 output MUX handles ALUOPC = 0–7 unchanged. Two multiplexers are added:

- MUX4 (2-to-1): bit 3 of ALUOPC selects between the original MUX result (bit 3 = 0) and the new logic-function output (bit 3 = 1).
- MUX5 (2-to-1): bit 0 of ALUOPC selects OR (ALUOPC = 8) or XOR (ALUOPC = 9).

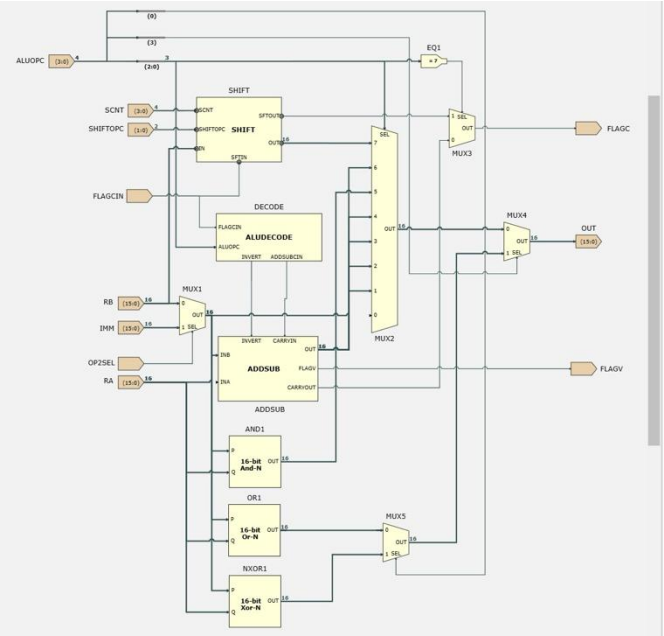
Step simulation confirms correct OR and XOR outputs; all original operations remain unaffected.

OR function:





XOR function:



The simulation uses the diagram as it was at the moment of pressing the "Start simulation" or "Refresh" button using default input values.

hex dec bin

Inputs

RA (16 bits)	b0000,0000,0101,0101
ALUOPC (4 bits)	b1000
SHIFTOPC (2 bits)	b00
OP2SEL	0
SCNT (4 bits)	b0000
FLAGCIN	0
RB (16 bits)	b0000,0000,1010,1010
IMM (16 bits)	b0000,0000,0000,0000

Outputs & Viewers

OUT (16 bits)	b0000,0000,1111,1111
FLAGC	0
FLAGV	0